Contributing

This is currently a solo personal project, but the guidelines below keep it consistent if that changes.

Before making a change

Read `docs/white-paper.md` first. It explains the math this project is built on and which parts are published versus assumed. Changes to the engine should either preserve the existing guarantees (Tier 1) or be explicit about which assumptions they're changing (Tier 2 / Daily Double). Every opponent-behavior value lives in the configuration block of `engine/behavior.py`, marked PUBLISHED or ASSUMED (ADR-0003).

Making a change

1. Change the code.
2. Update the relevant docs in the same pass: the white paper if the math changed, the ADRs if an assumption changed, the Diátaxis docs (`docs/tutorial/`, `docs/how-to/`, `docs/reference/`) if usage changed.
3. Add or update tests. Prefer hand-derived expected values (worked out on paper, not copied from the code's own output) wherever the math allows it. Where that's not possible (the Monte Carlo Daily Double engine), add invariant checks instead -- see `tests/test_dd_wager.py` for examples.
4. If you change the engine or `app/index.html`'s JavaScript, keep them in step (including the app's `BEHAVIOR` object, which mirrors `engine/behavior.py`) and run `node tests/js_parity.js`. CI runs it on every push and pull request -- see risk #8 in `RISK_REGISTER.md`.
5. Add a `CHANGELOG.md` entry: what changed, and why, following Keep a Changelog format, with a `### Record` section. Bump the version.
6. Regenerate any PDF in `build/` whose Markdown source changed: `python build/md_to_pdf.py <source.md> build/<name>.pdf`.
7. Run the test suite:

```
bash cd tests python -m pytest -v
```

Never do this

- Never embed raw rows from a scraped or restricted historical dataset in a shipped file -- see `docs/adr/0001-data-source-strategy.md`.
- Never silently rewrite a prior conclusion in the white paper or an ADR. Add new information and state what changed and why, so the history stays legible.
- Never present a Tier 2 / Daily Double win-probability number as a validated statistic. It is a model output that rests on the assumed values in `engine/behavior.py` (see ADR-0003).